Санкт-Петербургский государственный университет

Кафедра информационно-аналитических систем

Группа 23.Б08-мм

Реализация алгоритма поиска константных условных функциональных зависимостей CFDMiner

Смола Артём Михайлович

Отчёт по учебной практике
в форме «Решение»

Научный руководитель:
преподаватель кафедры ИАС, к. ф.-м. н., Чернышев Г. А.

Санкт-Петербург
2026

Оглавление

Введение **3**

1. Постановка задачи **5**

2. Обзор **6**

2.1. Предметная область 6

2.2. Алгоритм CFDMiner 10

2.3. Алгоритм GC-growth 13

2.4. Существующая реализация CFDMiner на платформе GitHub 17

3. Реализация CFDMiner **19**

3.1. Класс `CFDMiner` 19

3.2. Класс `GCGrowth` 20

4. Демонстрационный пример **23**

5. Использование ИИ-инструментов в работе **26**

6. Заключение **27**

Список литературы **28**

Введение

Наукоемкое профилирование данных позволяет обнаруживать неявные зависимости, взаимосвязи, закономерности и ограничения данных. Такие скрытые зависимости называются паттернами, а процесс их поиска — майнингом.

Одним из наиболее распространенных паттернов является функциональная зависимость (англ. Functional Dependency, FD). Для выполнения FD данные должны удовлетворять строгому ограничению: одинаковым значениям левой части должно соответствовать одно значение правой части. На практике более полезными могут оказаться зависимости, действующие только в определённом контексте. К таким паттернам относятся условные функциональные зависимости (англ. Conditional Functional Dependency, CFD) и их подкласс — константные условные функциональные зависимости (англ. Constant Conditional Functional Dependency, CCFD). CFD задаёт правило, связывающее значения атрибутов не для всей таблицы, а только для записей, удовлетворяющих определённым условиям. Константные CFD являются разновидностью CFD, в которых условия левой части и значение правой части заданы конкретными константами. Константные CFD могут быть особенно полезны в информационных системах, поскольку формализуют понятные для пользователя правила вида $(\text{Country} = \text{Russia} \wedge \text{PostIndex} = 101000) \rightarrow \text{City} = \text{Moscow}$. Данное правило означает, что для записи со страной “Russia” и почтовым индексом 101000 значением города должно быть “Moscow”.

К инструментам наукоемкого профилирования с широкими возможностями майнинга и валидации различных паттернов данных относится проект Desbordante [1]. На момент начала данной работы в Desbordante уже был реализован валидатор CFD, описанный в [7], а для поиска общих CFD был доступен алгоритм FD-First [5]. Его интеграция в проект и последующее улучшение описаны соответственно в [9] и [10]. Кроме того, велась интеграция алгоритма CFDFinder для гибкого поиска практически значимых CFD на основе статьи [3]. Со-

ответствующая реализация описана в [8]. Тем не менее, специализированного алгоритма поиска точных частых константных CFD в проекте не было. Поскольку алгоритм CFDMiner, представленный в статье [2], ориентирован именно на этот класс правил, его интеграция расширяет возможности проекта.

В данной работе предлагается реализовать CFDMiner в рамках проекта Desbordante.

1. Постановка задачи

Целью работы является реализация алгоритма CFDMiner обнаружения константных условных функциональных зависимостей в проекте Desbordante. Были поставлены следующие задачи.

1. Выполнить обзор алгоритма CFDMiner, используемого в нем алгоритма GC-growth, а также существующей реализации CFDMiner на платформе GitHub.
2. Реализовать алгоритм CFDMiner в проекте Desbordante.
3. Подготовить демонстрационный пример работы алгоритма.

2. Обзор

Цель данного раздела — изложить теоретические основы алгоритмов, используемых в работе. Определения даны на основе работ [2] и [6].

2.1. Предметная область

Определение 1. *Условной функциональной зависимостью (CFD) φ над схемой R называется $(X \to A, t_p)$, где*

- *X — набор атрибутов, A — единственный атрибут;*
- *$X \to A$ — стандартная FD (встроенная FD данной CFD);*
- *t_p — кортеж шаблонов из атрибутов $X \cup \{A\}$, в котором для $M \in X \cup \{A\}$, $t_p[M]$ либо константа из множества значений M, либо символ '_', которым обозначается, что условие по M выполняется для любого значения из множества значений M.*

Определение 2. *Левую часть X встроенной FD будем обозначать как* $\mathrm{LHS}(\varphi)$ *(англ. left-hand side), правую часть A —* $\mathrm{RHS}(\varphi)$ *(англ. right-hand side).*

Определение 3. *CFD $\varphi = (X \to A, t_p)$ называется константной, если t_p содержит только константы. В таком случае её удобно записывать как $(X \to A, (t_p \mid a))$, где t_p — набор констант для X, а a — константа для A.*

Определение 4. *CFD $\varphi = (X \to A, t_p)$ называется нетривиальной, если $A \notin X$.*

Определение 5. *Константная CFD $(X \to A, (t_p \mid a))$ выполняется на экземпляре отношения r, если*

$$r \models (X \to A, (t_p \mid a)) \quad \Longleftrightarrow \quad \forall t \in r\colon\ t[X] = t_p \Rightarrow t[A] = a.$$

Определение 6. *Константная CFD* $(X \to A, (t_p \mid a))$ *называется леворедуцированной на экземпляре отношения* r*, если*

$$\forall Y \subsetneq X: \quad r \not\models (Y \to A, (t_p[Y] \mid a)),$$

где $t_p[Y]$ *— ограничение набора констант* t_p *на атрибуты из* Y*.*

Пример. Пусть

$$r(A, B, C) = \{(a_1, b_1, c_1), (a_1, b_2, c_2), (a_2, b_1, c_2)\}.$$

CFD $(\{A, B\} \to C, ((a_1, b_1) \mid c_1))$ леворедуцирована: при удалении B её нарушает второй кортеж, а при удалении A — третий. Зависимость с пустой левой частью также не выполняется, так как в r встречается $C = c_2$.

Определение 7. *Константная CFD* φ *называется минимальной на экземпляре отношения* r*, если она нетривиальна, леворедуцирована на* r *и выполняется на* r*.*

Определение 8. *Поддержкой константной CFD* $\varphi = (X \to A, (t_p \mid a))$ *на экземпляре отношения* r *называется множество кортежей, совпадающих с её шаблоном:*

$$\mathrm{supp}(\varphi, r) = \{t \in r \mid t[X] = t_p \wedge t[A] = a\}.$$

Определение 9. *CFD* φ *называется* k*-частой, если* $|\mathrm{supp}(\varphi, r)| \geq k$*.*

Определение 10. *Пусть* r *— экземпляр отношения схемы* R*. Элементом транзакционной базы данных назовём пару* (A, a)*, где* $A \in R$ *и* a *— значение атрибута* A *в некотором кортеже из* r*. Множество всех таких элементов обозначим* $\mathcal{I}_r$*.*

Набором элементов называется произвольное подмножество $I \subseteq \mathcal{I}_r$*.*

Транзакцией, соответствующей кортежу $t \in r$*, называется набор*

$$\tau(t) = \{(A, t[A]) \mid A \in R\}.$$

Транзакционная база данных D, соответствующая r, — совокупность транзакций $\{\tau(t) \mid t \in r\}$.

Пример. Пусть отношение r состоит из двух кортежей:

	A	B
t_1	a_1	b_1
t_2	a_2	b_1

Этому отношению соответствуют транзакции $\tau(t_1) = \{(A, a_1), (B, b_1)\}$ и $\tau(t_2) = \{(A, a_2), (B, b_1)\}$, поэтому $D = \{\tau(t_1), \tau(t_2)\}$.

Определение 11. *Поддержкой набора элементов I в r называется множество кортежей, транзакции которых содержат I:*

$$\mathrm{supp}(I, r) = \{t \in r \mid I \subseteq \tau(t)\}.$$

Определение 12. *Набор I называется замкнутым, если не существует набора $J \supsetneq I$ с той же поддержкой.*

Пример. Пусть $r(A, B) = \{t_1 = (a_1, b_1), t_2 = (a_1, b_2)\}$. Набор $I = \{(A, a_1)\}$ замкнут: его поддержка равна $\{t_1, t_2\}$, а добавление элемента (B, b_1) или (B, b_2) уменьшает её до одного кортежа.

Определение 13. *Замыканием набора элементов I называется единственный максимальный по включению набор, содержащий I и имеющий ту же поддержку. Обозначение:* $\mathrm{clo}(I)$.

Определение 14. *Набор I называется открытым (или свободным), если не существует собственного подмножества $J \subsetneq I$ с той же поддержкой.*

Пример. Пусть $r(A, B) = \{t_1 = (a_1, b_1), t_2 = (a_1, b_2), t_3 = (a_2, b_1)\}$. Набор $I = \{(A, a_1), (B, b_1)\}$ свободен: его поддержка равна $\{t_1\}$, поддержки обоих одноэлементных подмножеств равны $\{t_1, t_2\}$ и $\{t_1, t_3\}$, а поддержка $\emptyset$ — все три кортежа.

Определение 15. *Свободный набор I, для которого* $\mathrm{clo}(I) = C$, *называется генератором замкнутого набора C.*

Определение 16. *Зафиксируем полный порядок $\prec_0$ на элементах множества $\mathcal{I}_r$.*

Для наборов элементов $X \subseteq Y$ будем говорить, что X является префиксом Y, если каждый элемент из X предшествует по порядку $\prec_0$ каждому элементу из $Y \setminus X$:

$$\forall x \in X \; \forall y \in Y \setminus X \colon \; x \prec_0 y.$$

Последний по порядку $\prec_0$ элемент непустого набора X обозначим $\mathrm{last}(X)$.

Порядок $\prec_0$ на элементах индуцирует лексикографический порядок $\prec_{\mathrm{lex}}$ на наборах элементов.

Деревом перечисления наборов называется корневое дерево всех подмножеств $\mathcal{I}_r$. Его корень соответствует пустому набору, а дочерними вершинами корня являются одноэлементные наборы.

Пусть $\mathrm{parent}(X)$ *обозначает родителя вершины X. Для непустой вершины X определим множество её последующих соседей:*

$$\mathrm{Right}(X) = \{Y \mid \mathrm{parent}(Y) = \mathrm{parent}(X) \wedge X \prec_{\mathrm{lex}} Y\}.$$

Дочерними вершинами X являются наборы

$$\mathrm{Child}(X) = \{X \cup Y \mid Y \in \mathrm{Right}(X)\}.$$

Обход дерева определяется рекурсивно: сначала посещается вершина X, после чего рекурсивно обходятся поддеревья в порядке, обратном $\prec_{\mathrm{lex}}$ (справа налево).

Порядок $\prec_1$ на вершинах дерева задаётся этим обходом: $X \prec_1 Y$, если вершина X посещается раньше вершины Y.

Пример. Пусть $\mathcal{I}_r = \{a, b, c\}$ и $a \prec_0 b \prec_0 c$. Тогда $\{a\}$ является префиксом $\{a, c\}$, $\mathrm{last}(\{a, c\}) = c$, $\mathrm{Right}(\{a\}) = \{\{b\}, \{c\}\}$ и $\mathrm{Child}(\{a\}) = \{\{a, b\}, \{a, c\}\}$. Обход всего дерева справа налево посещает вершины в

порядке

$$\emptyset,\ \{c\},\ \{b\},\ \{b,c\},\ \{a\},\ \{a,c\},\ \{a,b\},\ \{a,b,c\}.$$

Определение 17. *Префиксным деревом транзакционной базы данных D называется корневое дерево, множество вершин которого равно*

$$V_D = \{X \mid \exists T \in D\colon X \text{ является префиксом } T\}.$$

Корень соответствует пустому набору. Для каждой непустой вершины X родителем является набор

$$\text{parent}(X) = X \setminus \{\text{last}(X)\}.$$

Определение 18. *Вершина X называется терминальной, если существует транзакция $T \in D$, для которой $X = T$.*

Определение 19. *Пусть D — транзакционная база данных. Фильтром набора элементов P будем называть $f(P, D) = \{T \in D \mid P \subseteq T\}$. Для транзакционной базы данных D, соответствующей экземпляру отношения r, выполняется $|f(P, D)| = |\,\text{supp}(P, r)|$.*

Определение 20. *Введем следующее отношение эквивалентности $\sim_D$: $P \sim_D Q \iff f(P, D) = f(Q, D)$. Эквивалентностный класс P по $\sim_D$ будем обозначать $[P]_D$.*

Пример. Пусть $r(A, B, C) = \{t_1 = (a_1, b_1, c_1), t_2 = (a_1, b_1, c_2), t_3 = (a_2, b_2, c_1)\}$ и $D = \{\tau(t_1), \tau(t_2), \tau(t_3)\}$. Тогда $P = \{(A, a_1)\}$ и $Q = \{(B, b_1)\}$ эквивалентны, поскольку $f(P, D) = f(Q, D) = \{\tau(t_1), \tau(t_2)\}$.

2.2. Алгоритм CFDMiner

CFDMiner [2] предназначен для поиска канонического покрытия k-частых минимальных константных CFD. На вход алгоритм принимает экземпляр отношения r схемы R и порог минимальной поддержки k.

Основная часть обработки в CFDMiner начинается после построения отображения *C2F*, сопоставляющего каждому k-частому замкну-

тому набору множество соответствующих ему k-частых свободных наборов. В исходной статье [2] этот подготовительный этап выполняется алгоритмом GC-growth, описанным в [6].

2.2.1. Ключевое утверждение

CFDMiner основан на связи между леворедуцированными константными CFD, свободными и замкнутыми наборами. В статье [2] формулируется и доказывается следующее утверждение.

Утверждение 1. *Пусть r — экземпляр отношения схемы R, а $\varphi = (X \to A, (t_p \mid a))$ — нетривиальная константная CFD. Зависимость φ является k-частой, леворедуцированной и выполняется на r тогда и только тогда, когда выполнены следующие условия:*

1. *(X, t_p) является свободным, k-частым и не содержит (A, a);*
2. *$\mathrm{clo}(X, t_p)$ содержит (A, a);*
3. *не существует свободного набора (Y, s_p), являющегося собственным подмножеством (X, t_p), для которого $\mathrm{clo}(Y, s_p)$ содержит (A, a).*

2.2.2. Шаги алгоритма

Псевдокод CFDMiner приведён в алгоритме 1. Сначала строится отображение *C2F*, создаются пустая хеш-таблица H, сопоставляющая свободным наборам множества кандидатов правой части, и список свободных наборов L (строки 3–6). Затем алгоритм обходит замкнутые наборы C из *C2F* и соответствующие им свободные наборы F (строки 7–15). При первом появлении F он добавляется в H и L. После этого в $H[F]$ добавляется разность $C \setminus F$, то есть элементы замыкания F, которые могут образовать правую часть зависимости.

Список L упорядочивается по возрастанию размера набора, после чего обрабатывается каждый свободный набор F. Из $H[F]$ исключаются кандидаты, уже полученные для его собственных свободных подмножеств $G \subsetneq F$ (строки 18–22). Тем самым удаляются зависимости с

избыточной левой частью. Для каждого оставшегося элемента $q \in H[F]$ в результат добавляется константная CFD $F \to q$ (строки 23–25).

В работе [4] предложена оптимизация CFDMiner, при которой на этапе фильтрации рассматриваются только непосредственные подмножества $F \setminus \{x\}$ свободного набора F, а не все его непустые собственные подмножества. Число проверок уменьшается с $2^{|F|} - 2$ до $|F|$ без изменения результирующего канонического покрытия.

Алгоритм 1 CFDMiner

1: **Вход:** экземпляр отношения r, минимальная поддержка k
2: **Выход:** каноническое покрытие Σ k-частых минимальных константных CFD
3: $C2F \leftarrow \mathrm{GCGrowth}(r, k)$
4: $H \leftarrow \emptyset$
5: $L \leftarrow \emptyset$
6: $\Sigma \leftarrow \emptyset$
7: **for** $C \in C2F$ **do**
8: **for** $F \in C2F[C]$ **do**
9: **if** $F \notin H$ **then**
10: $H[F] \leftarrow \emptyset$
11: $L \leftarrow L \cup \{F\}$
12: **end if**
13: $H[F] \leftarrow H[F] \cup (C \setminus F)$
14: **end for**
15: **end for**
16: упорядочить L по возрастанию размера набора
17: **for** $F \in L$ **do**
18: **for** $G \subsetneq F$ **do**
19: **if** $G \in H$ **then**
20: $H[F] \leftarrow H[F] \setminus H[G]$
21: **end if**
22: **end for**
23: **for** $q \in H[F]$ **do**
24: $\Sigma \leftarrow \Sigma \cup \{F \to q\}$
25: **end for**
26: **end for**

Пример. Рассмотрим некоторые шаги работы алгоритма 1 на следующем отношении при $k = 2$:

	A	B	C	D	E
t_1	a_1	b_1	c_1	d_1	e_1
t_2	a_1	b_1	c_1	d_1	e_2
t_3	a_1	b_2	c_1	d_2	e_1
t_4	a_2	b_1	c_2	d_2	e_2

Рассмотрим свободные наборы $G = \{(A, a_1)\}$ и $F = \{(A, a_1), (B, b_1)\}$: их поддержки равны $\{t_1, t_2, t_3\}$ и $\{t_1, t_2\}$ соответственно. Их замыкания имеют вид $C_G = G \cup \{(C, c_1)\}$ и $C_F = F \cup \{(C, c_1), (D, d_1)\}$.

При заполнении H в строках 7–15 из этих записей получаем $H[G] = C_G \setminus G = \{(C, c_1)\}$ и $H[F] = C_F \setminus F = \{(C, c_1), (D, d_1)\}$. После сортировки L набор G обрабатывается раньше F, и в Σ добавляется CFD $(A \to C, (a_1 \mid c_1))$. Поэтому при обработке F в строке 20 H меняется следующим образом:

$$H[F] = \{(C, c_1), (D, d_1)\} \quad \longrightarrow \quad H[F] = H[F] \setminus H[G] = \{(D, d_1)\}.$$

У остальных собственных подмножеств F, то есть $\emptyset$ и $\{(B, b_1)\}$, множества кандидатов пусты. Поэтому строки 23–25 добавляют CFD $(\{A, B\} \to D, ((a_1, b_1) \mid d_1))$ с поддержкой $\{t_1, t_2\}$. Удалить A из её левой части не позволяет t_4, а удалить B — t_3.

2.3. Алгоритм GC-growth

Алгоритм GC-growth [6] находит k-частые свободные и замкнутые наборы и связывает их.

2.3.1. Вспомогательные структуры и обозначения

В алгоритме используются следующие вспомогательные структуры и обозначения:

- основной структурой, хранящей транзакции служит префиксное

дерево (в статье [2] без отдельного обозначения);

- булево отображение $T[\cdot]$: значение $T[X]$ равно true, если X является терминальной вершиной;
- булево отображение $G[\cdot]$: значение $G[X]$ равно true, если X является установленным генератором;
- отображение last, сопоставляющее каждому непустому набору его последний по порядку $\prec_0$ элемент;
- отображение $H[\cdot]$, которое каждому элементу $a \in \mathcal{I}_r$ сопоставляет вершины префиксного дерева, чьи пути оканчиваются элементом a;
- отображение $P[\cdot]$. Значение $P[X]$ равно числу кортежей $t \in r$, для которых X является префиксом $\tau(t)$;
- отображение $R[\cdot]$, сопоставляющее замкнутому набору множество его генераторов (отображение *C2F* в терминологии CFDMiner);
- отображение $S[\cdot]$, сопоставляющее кандидату временное значение общей поддержки.

2.3.2. Ключевые утверждения

Далее приведены основные утверждения, важные для алгоритма. Доказательства данных утверждений содержатся в статье [6].

Утверждение 2. *Для набора элементов X его поддержка равна*

$$|f(X, D)| = \sum_{\substack{X' \in H[\text{last}(X)] \\ X \subseteq X'}} P[X'].$$

Утверждение 3. *Пусть все непосредственные подмножества набора элементов X уже обработаны. Тогда X является k-частым генератором тогда и только тогда, когда выполнены следующие условия:*

1. $|f(X, D)| \geq k$;

2. для каждого $\alpha \in X$ выполнено

$$|f(X, D)| < |f(X \setminus \{\alpha\}, D)|;$$

3. каждое множество $X \setminus \{\alpha\}$, где $\alpha \in X$, является генератором.

Утверждение 4. *Если X — генератор в D, то замкнутый набор его класса эквивалентности равен*

$$\mathrm{clo}(X) = \bigcap \mathcal{T}_X,$$

где $\mathcal{T}_X$ — множество таких терминальных вершин X'', что существует набор $X' \in H[\mathrm{last}(X)]$, для которого $X \subseteq X'$ и X' является префиксом X''.

По сути, утверждение 4 формулирует следующее: пересечение терминальных вершин, содержащих в качестве префикса подходящие элементы из $H[\mathrm{last}(X)]$ (вместо самого X), в точности даёт замыкание X.

2.3.3. Шаги алгоритма

Псевдокод GC-GROWTH находится в алгоритме 2. Утверждение 2 используется на седьмой строке псевдокода. В условии на включение в набор генераторов на восьмой строке используется утверждение 3. Для вычисления замыкания генератора на десятой строке используется утверждение 4.

Алгоритм 2 GC-GROWTH

1: **Вход:** транзакционная база данных D, порог поддержки k
2: **Выход:** генераторы, замкнутые наборы и их поддержки
3: одним проходом по D заполнить $P[\cdot]$, $T[\cdot]$ и $H[\cdot]$
4: $G[\emptyset] \leftarrow$ true; заполнить $R[\cdot]$ пустыми значениями
5: упорядочить вершины дерева перечисления наборов $X_1, \ldots, X_n$ так, что $X_1 \prec_1 \ldots \prec_1 X_n$
6: **for** $i = 1, \ldots, n$ **do**
7: $\quad S[X_i] \leftarrow \sum_{X \in H[\text{last}(X_i)],\, X_i \subseteq X} P[X]$
8: $\quad$ **if** $S[X_i] \geq k \wedge \forall \alpha \in X_i\colon\ S[X_i] < S[X_i \setminus \{\alpha\}] \wedge \forall \alpha \in X_i\colon\ G[X_i \setminus \{\alpha\}] =$ true **then**
9: $\quad\quad G[X_i] \leftarrow$ true
10: $\quad\quad C \leftarrow \text{clo}(X_i)$
11: $\quad\quad R[C] \leftarrow R[C] \cup \{X_i\}$
12: $\quad\quad S[C] \leftarrow S[X_i]$
13: $\quad$ **else**
14: $\quad\quad G[X_i] \leftarrow$ false
15: $\quad$ **end if**
16: **end for**
17: **for** каждого C, для которого $R[C] \neq \emptyset$ **do**
18: $\quad$ вывести $R[C]$, C и $S[C]$
19: **end for**

Пример. Рассмотрим работу алгоритма 2 на следующем отношении при $k = 2$:

	A	B	C	D
t_1	a_1	b_1	c_1	d_1
t_2	a_1	b_1	c_1	d_2
t_3	a_1	b_2	c_2	d_1
t_4	a_2	b_1	c_2	d_2

Зафиксируем порядок $\prec_0$, в котором сначала идут элементы атрибута A, затем B, C и D; внутри каждого атрибута индекс 1 предшествует индексу 2. Для $X = \{(A, a_1), (B, b_1)\}$ имеем $H[(B, b_1)] = \{X, Z\}$, где $Z = \{(A, a_2), (B, b_1)\}$, причём $P[X] = 2$, $P[Z] = 1$. В строке 7 учитывается только вершина X, поскольку $X \not\subseteq Z$, и получается $S[X] = P[X] = 2$.

К этой итерации оба непосредственных подмножества X уже отмечены как генераторы, и $S[\{(A, a_1)\}] = S[\{(B, b_1)\}] = 3$. Следовательно, в

строке 8 выполнены все условия: $2 \geq k$, $2 < 3$ для обоих подмножеств и оба значения G равны true. Строки 9–10 устанавливают $G[X] = \text{true}$ и находят $C = \tau(t_1) \cap \tau(t_2) = \{(A, a_1), (B, b_1), (C, c_1)\}$. При выбранном порядке генератор $\{(C, c_1)\}$ этого замыкания уже найден, поэтому строка 11 обновляет

$$R[C] = \big\{\{(C, c_1)\}\big\} \quad \longrightarrow \quad R[C] = R[C] \cup \{X\} = \big\{\{(C, c_1)\}, X\big\},$$

а строка 12 сохраняет $S[C] = 2$.

Для сравнения, у набора $Y = \{(A, a_1), (C, c_1)\}$ поддержка также равна $\{t_1, t_2\}$, но $S[Y] = S[\{(C, c_1)\}] = 2$. Строгое неравенство в строке 8 нарушено, поэтому строка 14 задаёт $G[Y] = \text{false}$ и на этой итерации R не пополняется.

2.4. Существующая реализация CFDMiner на платформе GitHub

На платформе GitHub был найден репозиторий с реализацией алгоритма обнаружения константных CFD на языке C++[1]. Его автор отмечает, что программа является его интерпретацией алгоритма CFDMiner, а не воспроизведением исходной реализации.

В статье алгоритм CFDMiner получает отображение *C2F* как результат отдельного алгоритма поиска замкнутых наборов и генераторов, а затем строит каноническое покрытие CFD. В репозитории вместо этого реализован собственный рекурсивный обход наборов элементов: одновременно строятся генераторы, их замыкания и итоговые зависимости.

Анализ реализации выявил два случая неполноты результата.

1. Реализация начинает поиск с одноэлементных наборов и не сохраняет пустой набор как генератор. Поэтому для отношения $r_1(A, B) = \{(1, 0), (1, 1), (1, 0)\}$ при $k = 3$ она не выводит зависимость $\emptyset \to A$, $(\emptyset \mid 1)$, хотя она точна и k-часта.

[1]URL: https://github.com/j-r77/cfdminer

2. Параметр `maxsize` ограничивает глубину рекурсивного обхода, а не только размер левой части выводимой CFD. Для отношения из таблицы 1 при $k = 2$ и `maxsize`=2 не выводится минимальная CFD $(B = 0, C = 0) \to A = 1$ с поддержкой 2. Она появляется лишь при `maxsize`=3, хотя размер её левой части равен двум.

Таблица 1: Входные данные второго контрпримера

	A	B	C
t_1	1	0	0
t_2	0	0	1
t_3	1	0	1
t_4	1	0	0
t_5	0	1	0
t_6	1	1	0

В итоге было решено не использовать данную реализацию в качестве baseline для реализации CFDMiner в Desbordante.

3. Реализация CFDMiner

Реализация CFDMiner разделяет построение отображения *C2F* и формирование результирующего набора CFD. Класс `CFDMiner` отвечает за настройку алгоритма и регистрацию найденных зависимостей, а класс `GCGrowth` — за построение отображения *C2F*. Диаграмма классов архитектуры реализации CFDMiner приведена на Рис. 1.

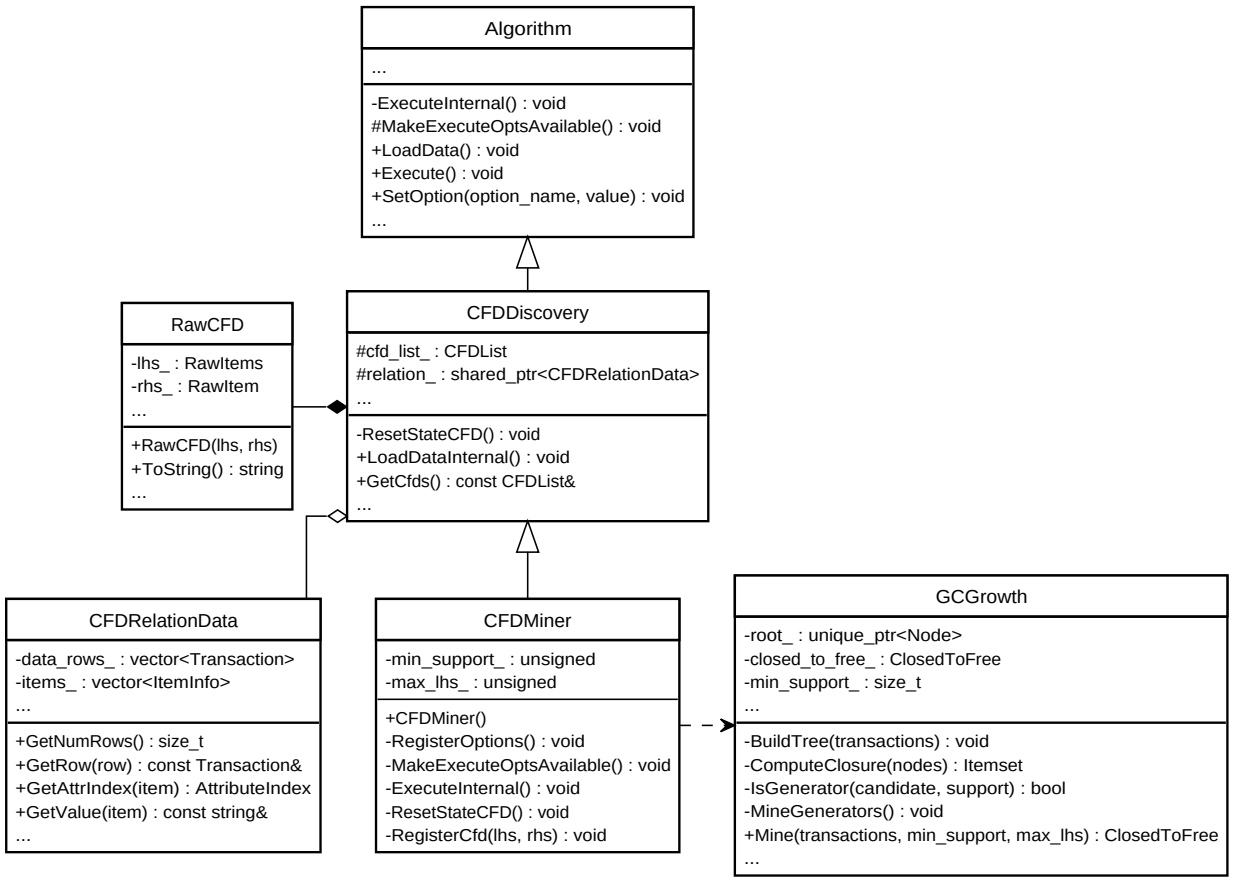


Рис. 1: Диаграмма классов реализации алгоритма CFDMiner

3.1. Класс `CFDMiner`

В конструкторе класса вызывается метод `RegisterOptions()`, в котором регистрируются две опции алгоритма: минимальный порог поддержки (поле `min_support_`) и максимальный размер левой части CFD (`max_lhs_`).

Метод `ExecuteInternal()` Основная логика псевдокода из статьи [2] реализована в методе `ExecuteInternal()`. Для получения отображения *C2F* создается объект типа `GCGrowth` и вызывается его метод `Mine()`. Отображение $H[\cdot]$ и список L из алгоритма 1 представлены объектом `generator_to_closure` типа `std::map`, который обеспечивает упорядоченное хранение ключей. На этапе удаления из правой части кандидатов замыканий подмножеств реализуется оптимизация с рассмотрением только непосредственных подмножеств кандидатов. Для заполнения списка CFD реализован метод `RegisterCfd()`.

Набор CFD сбрасывается перед запуском алгоритма методом `ResetState()` родительского класса `CFDDiscovery`.

3.2. Класс `GCGrowth`

Метод `Mine()` Основным методом класса `GCGrowth` является метод `Mine()`, возвращающий объект типа `ClosedToFree` (представление отображения *C2F* в коде). В нем вызывается метод `BuildTree()` построения префиксного дерева. Затем происходит добавление в результат пустого генератора и запуск метода `MineGenerators()` с обходом префиксного дерева и построением основной части результата.

Метод `MineGenerators()` В методе `MineGenerators()` определяется вспомогательная встроенная структура `GrowthTask`. `GrowthTask` содержит три значения:

- `prefix` — построенный набор без очередного добавляемого элемента (соответствует некоторому пройденному X_i в псевдокоде);
- `extension_index` — индекс элемента, который будет добавлен к набору слева;
- `matching_nodes` — множество вершин префиксного дерева, оканчивающихся на тот же элемент, что и `prefix`, и пути к которым содержат `candidate`, где `candidate` получается добавлением к пре-

фиксу элемента с индексом `extension_index` (соответствует множеству X' в терминах утверждения 4).

Добавление элемента в `prefix` именно слева обеспечивает то, что `matching_nodes` расширений является подмножеством `matching_nodes` расширяемого набора.

Стек `tasks` элементов типа `GrowthTask` задаёт множество и порядок обхода наборов. Изначально в `tasks` добавляются все задачи соответствующие тройкам: $\{\emptyset, a_i, H[a_i]\}$, где первый элемент соответствует пустому `prefix`, a_i — один из элементов из множества всех k-частых элементов таблицы, $H[a_i]$ — множество вершин префиксного дерева, оканчивающихся на a_i.

Далее в цикле `while (!tasks.empty())` на каждой итерации снимается вершина стека `tasks`. Внутри цикла:

- формируется `candidate` добавлением к `prefix` слева элемента соответствующего `extension_index`;
- вычисляется поддержка `candidate` согласно утверждению 2;
- выполняется проверка, является ли набор `candidate` генератором согласно утверждению 3; если условия не выполнены, то выполняется переход к новой итерации цикла;
- полученный генератор и его поддержка добавляются в отображение `gen_to_supp_` типа `std::unordered_map` (соответствует структурам $G[\cdot]$ и $S[\cdot]$ псевдокода);
- результат пополняется найденным генератором; ключом отображения является замыкание генератора, вычисляемое методом `ComputeClosure`;
- если длина `candidate` строго меньше `max_lhs_`, то выполняется построение множества расширений с использованием метода `FindExtensions()`;

- полученные расширения `candidate` добавляются в `tasks` и происходит переход к следующей итерации.

`FindExtensions()` расширяет текущий `candidate` только элементами, которые являются метками предка вершины из `matching_nodes` и чей индекс строго меньше `extension_index`. Метод `FindExtensions()` возвращает упорядоченное отображение `std::map`, сопоставляющее каждому индексу расширяющего элемента соответствующее подмножество `matching_nodes`.

4. Демонстрационный пример

Для демонстрации работы реализации CFDMiner подготовлен пример на языке python, использующий интерфейс проекта Desbordante. Пример показывает загрузку табличных данных, задание минимальной поддержки и максимального размера левой части CFD, запуск алгоритма и получение найденных зависимостей.

В примере рассматриваются два набора данных. На данных о позициях в игре «крестики-нолики» демонстрируется обнаружение условий победы и влияние ограничения на размер левой части зависимости. На наборе данных Zoo показана интерпретация найденных зависимостей между признаками животных и их классами. Вывод программы приведён на рисунках 2 и 3.

```
================================================================================
Поиск точных константных CFD с помощью CFDMiner
================================================================================
Константная условная функциональная зависимость (constant CFD) имеет вид (X -> A, (t_p |
a)). Здесь X -> A - встроенная функциональная зависимость, лежащая в основе правила, t_p
- набор констант для X, а a - константа для A. Во всех строках, где значения столбцов X
совпадают с t_p, значение столбца A должно быть равно a. Например, ([Milk] -> Class,
(yes | mammal)) означает: если Milk=yes, то Class=mammal.

Авторы алгоритма CFDMiner: W. Fan, F. Geerts, J. Li и M. Xiong. Алгоритм сначала находит
частые свободные наборы элементов и их замыкания, а затем строит по ним минимальные
константные CFD. Алгоритм ищет только точные зависимости: среди строк, к которым
применима левая часть найденной CFD, не должно быть ни одного нарушения.

Параметры алгоритма:
  cfd_minsup   - минимальная поддержка, заданная числом строк
  cfd_max_lhs  - максимальное число элементов в левой части CFD

Поддержка CFD - это число строк таблицы, которые ей удовлетворяют. Чем выше cfd_minsup,
тем больше подтверждающих строк требуется правилу и тем меньше правил останется в
результате. Увеличение cfd_max_lhs позволяет учитывать более сложные сочетания
признаков, но расширяет пространство поиска и обычно приводит к росту числа найденных
правил.

================================================================================
Наборы данных
================================================================================
Tic-Tac-Toe Endgame содержит 138 записей и 10 столбцов. Каждая строка описывает конечную
позицию на поле 3x3, а столбец X_Wins показывает, победили ли "крестики". Позиции,
которые переходят друг в друга при повороте или отражении поля, объединены, поэтому в
наборе осталось 138 различных конфигураций.

Набор Zoo состоит из 101 строки и 17 столбцов. В них записаны признаки животных и один
из семи биологических классов.

================================================================================
Сценарий 1. Поиск выигрышных комбинаций
================================================================================
Чтобы описать выигрышный ряд, нужны значения трёх клеток. Поэтому разрешим до трёх
элементов в левой части CFD и установим минимальную поддержку 10. После запуска CFDMiner
рассмотрим зависимости, в правой части которых указано X_Wins=yes.

Всего CFDMiner нашёл 49 CFD.
В 16 из них правая часть определяет результат игры.
Ниже приведены четыре частых правила, соответствующих победе "крестиков":
  (Bottom_Left=x, Bottom_Middle=x, Bottom_Right=x) -> (X_Wins=yes) [поддержка: 27]
  (Middle_Left=x, Center=x, Middle_Right=x) -> (X_Wins=yes) [поддержка: 23]
  (Top_Right=x, Middle_Right=x, Bottom_Right=x) -> (X_Wins=yes) [поддержка: 16]
  (Top_Right=x, Center=x, Bottom_Left=x) -> (X_Wins=yes) [поддержка: 28]

Алгоритм в некотором смысле "восстанавливает" суть игры из набора.

================================================================================
Сценарий 2. Выбор ограничения на размер левой части
================================================================================
```

Рис. 2: Результат выполнения демонстрационного примера, первая часть

```
Параметр cfd_max_lhs влияет не только на время работы: от него зависит, какие
закономерности алгоритм в принципе сможет выразить. Победная комбинация занимает три
клетки, поэтому при ограничении в два элемента полностью описать её невозможно.

  cfd_max_lhs=2: всего 15 CFD, CFD для X_Wins: 0
  cfd_max_lhs=3: всего 49 CFD, CFD для X_Wins: 16

При cfd_max_lhs=2 алгоритм не находит ни одного точного правила для X_Wins с поддержкой
не менее 10.

================================================================================
Сценарий 3. Разбор биологических закономерностей
================================================================================
Для набора Zoo ограничим левую часть одним элементом и зададим минимальную поддержку 20.
Чтобы их было проще интерпретировать, рассмотрим несколько наиболее показательных
зависимостей, объединив их в группы.

Всего CFDMiner обнаружил 54 CFD.
Два взаимно обратных правила для млекопитающих:
  (Milk=yes) -> (Class=mammal) [поддержка: 41]
  (Class=mammal) -> (Milk=yes) [поддержка: 41]

Поддержка каждого правила равна 41. Иными словами, в наборе Zoo строки с Milk=yes в
точности совпадают со строками, где Class=mammal.

Рассмотрим зависимости, описывающие птиц:
  (Feathers=yes) -> (Class=bird) [поддержка: 20]
  (Class=bird) -> (Feathers=yes) [поддержка: 20]
  (Feathers=yes) -> (Eggs=yes) [поддержка: 20]
  (Feathers=yes) -> (Legs=2) [поддержка: 20]

В Zoo все животные с Feathers=yes относятся к классу bird, а у всех птиц есть перья: обе
зависимости описывают одну и ту же группу из 20 строк. Кроме того, для каждого животного
из этой группы Eggs=yes и Legs=2. Таким образом, найденные правила описывают сразу
несколько общих признаков.

Закономерности, не привязанные к одному классу:
  (Toothed=yes) -> (Backbone=yes) [поддержка: 61]
  (Airborne=yes) -> (Breathes=yes) [поддержка: 24]

Пример показывает, что полезная CFD на данном наборе не обязательно должна содержать
столбец Class. Например, зубастые животные относятся к разным классам, но во всех 61
соответствующих строках Backbone=yes. Аналогично, во всех 24 строках с Airborne=yes
выполняется Breathes=yes. Обратные зависимости верны уже не для всех строк.

================================================================================
Итоги
================================================================================
CFDMiner подходит для поиска точных зависимостей, связанных с конкретными значениями.
Параметр cfd_minsup задаёт необходимый объём подтверждающих данных, а cfd_max_lhs
ограничивает сложность контекста. Найденные зависимости удобнее анализировать не по
отдельности, а группами и с учётом предметной области. Если нужны CFD с символами
подстановки или правила, допускающие нарушения, используйте другие алгоритмы из
examples/basic/mining_cfd.py.
```

Рис. 3: Результат выполнения демонстрационного примера, продолжение

5. Использование ИИ-инструментов в работе

В работе ИИ-инструменты использовались для следующих задач: ревью и улучшение кода, проверка текста на наличие орфографических, стилистических и логических ошибок, создание примеров и пояснений к определениям и алгоритмам. Применялись языковые модели семейства OpenAI GPT в рамках агентной среды OpenAI Codex.

6. Заключение

1. Изучены теоретические основы обнаружения константных CFD, алгоритмы CFDMiner и GC-growth. Проведён анализ реализации CFDMiner на платфоре GitHub. Установлено, что она отличается от алгоритма статьи и в некоторых граничных случаях возвращает неполный набор зависимостей.

2. В проект Desbordante интегрирован алгоритм CFDMiner на основе статей [2] и [6].

3. Подготовлен демонстрационный пример работы алгоритма.

Ссылка на pull request: https://github.com/Desbordante/desbordante-core/pull/840.

Список литературы

[1] Desbordante: from benchmarking suite to high-performance science-intensive data profiler / George Chernishev, Michael Polyntsov, Anton Chizhov et al. // Proceedings of the 8th International Conference on Data Science and Management of Data (12th ACM IKDD CODS and 30th COMAD). — CODS-COMAD '24. — New York, NY, USA : Association for Computing Machinery, 2025. — P. 234–243. — URL: https://doi.org/10.1145/3703323.3703725.

[2] Discovering Conditional Functional Dependencies / Wenfei Fan, Floris Geerts, Jianzhong Li, Ming Xiong // IEEE Transactions on Knowledge and Data Engineering. — 2011. — Vol. 23, no. 5. — P. 683–698.

[3] Grundke Maximilian. Discovering Interesting Conditional Functional Dependencies. — 2018.

[4] An Optimization Strategy for CFDMiner: An Algorithm of Discovering Constant Conditional Functional Dependencies / Jinling Zhou, Xingchun Diao, Jianjun Cao, Zhisong Pan // IEICE TRANSACTIONS on Information. — 2016. — February. — Vol. E99-D, no. 2. — P. 537–540.

[5] Rammelaere Joeri, Geerts Floris. Revisiting Conditional Functional Dependency Discovery: Splitting the "C" from the "FD" // Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2018, Dublin, Ireland, September 10–14, 2018, Proceedings, Part II. — Berlin, Heidelberg : Springer-Verlag, 2018. — P. 552–568. — URL: https://doi.org/10.1007/978-3-030-10928-8_33.

[6] Relative risk and odds ratio: a data mining perspective / Haiquan Li, Jinyan Li, Limsoon Wong et al. — 2005. — P. 368–377. — URL: https://doi.org/10.1145/1065167.1065215.

[7] Дмитрий Федосеев. Реализация валидатора условных функциональных зависимостей в проекте Desbordante. — 2024. — URL: https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/CFD%20Verification%20-%20Dmitry%20Fedoseev%20-%202024%20autumn.pdf.

[8] Иван Кожуков. Реализация алгоритма поиска условных функциональных зависимостей CFDFinder. — 2025. — URL: https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/CFDFINDER%20Implementation%20-%20Kozhukov%20Ivan%20-%202025%20spring.pdf.

[9] Михаил Выродов. Интеграция алгоритма по поиску условных функциональных зависимостей в "Desbordante". — 2022. — URL: https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/CFD%20FD-First%20Integration%20-%20Mikhail%20Vyrodov%20-%202022%20autumn.pdf.

[10] Михаил Выродов. Улучшение кода алгоритма по поиску условных функциональных зависимостей в "Desbordante". — 2023. — URL: https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/CFD%20FD-First%20Optimization%20-%20Mikhail%20Vyrodov%20-%202023%20spring.pdf.